EXPERIMENT - 5

Aim : Hands on Solidity Programming Assignments for creating Smart Contracts

Lab Objectives : To explore Blockchain concepts.

Lab Outcomes (LO) : Design Smart Contract using Solidity (LO2)

Theory :

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development.
Commonly used types include :

- uint / int: unsigned and signed integers of different sizes (e.g., uint256, int128).
- bool: represents logical values (true or false).
- address: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- bytes / string: store binary data or textual data.

Variables in Solidity can be state variables (stored on the blockchain permanently), local variables (temporary, created during function execution), or global variables (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:
● pure: cannot read or modify blockchain state; they work only with inputs and internal computations.
● view: can read state variables but cannot alter them. This classification helps optimize gas usage and enforce function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values.
Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation.
For example, a function can accept an amount in Ether and return whether the transfer was successful.
Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

● Function Visibility defines who can access a function:
o public: available both inside and outside the contract.
o private: only accessible within the same contract.
o internal: accessible within the contract and its child contracts.
o external: can be called only by external accounts or other contracts.

● Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
● Constructors are special functions executed only once during contract deployment.

They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

● if-else allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.

● Loops (for, while, do-while) enable repeated execution of code.

For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

● Arrays: Can be fixed or dynamic and are used to store ordered lists of elements.

Example: an array of addresses for registered users.

● Mappings: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.

● Structs: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.

● Enums: Used to define a set of predefined constants, making code more readable.

Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

● storage: Data stored permanently on the blockchain. Examples: state variables.

● memory: Temporary data storage that exists only while a function is executing.

Used for local variables and function inputs.

● calldata: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory.

Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

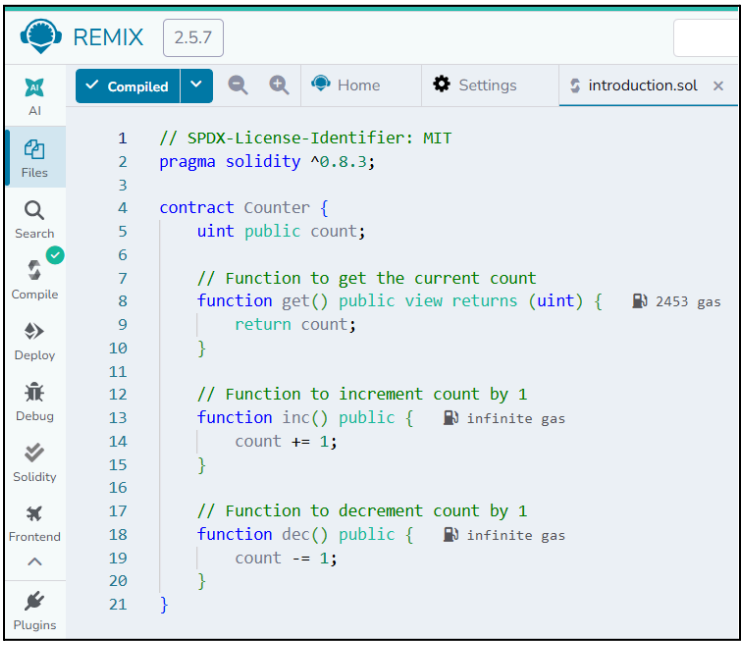
● Ether and Wei: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 1018 Wei). This ensures high precision in financial transactions.

● Gas and Gas Price: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.

● Sending Transactions: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design is very important.

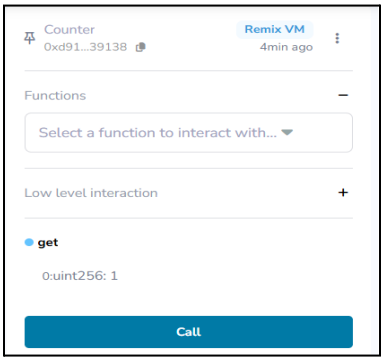
Implementation :

Tutorial no. 1 – Compile the code



Compiled

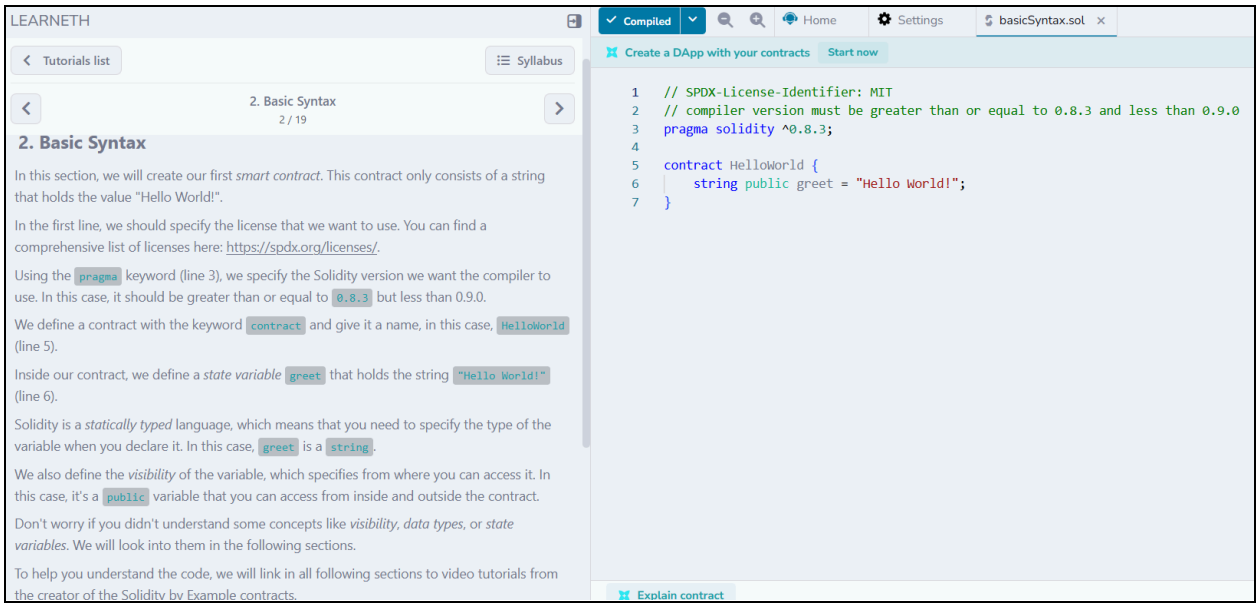
Deploy the contract



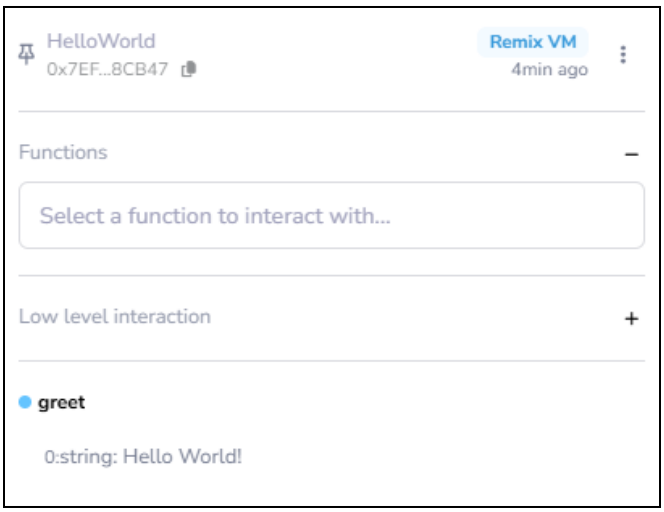
get → 0 inc → click get → 1 inc → click get → 2 dec → click get → 1



Tutorial no. 2

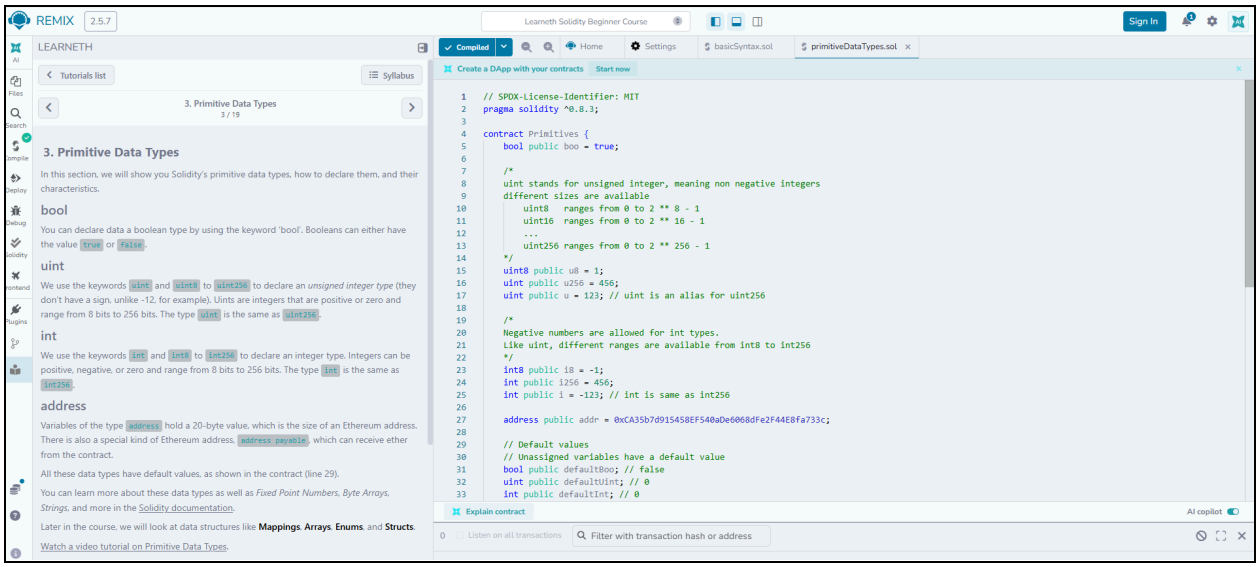


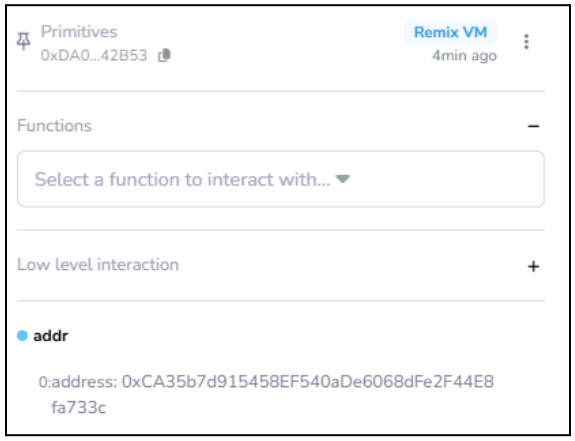
Compiled



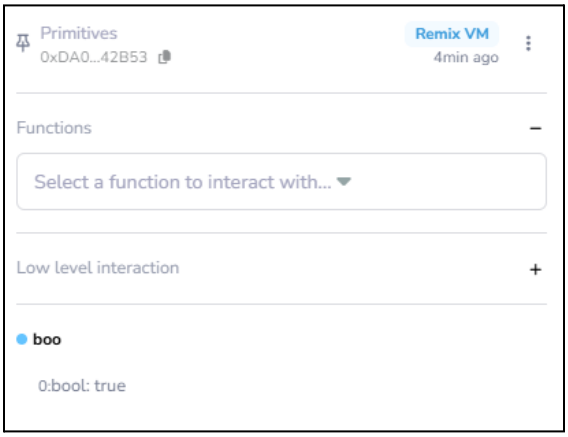
Deployed

Tutorial no. 3

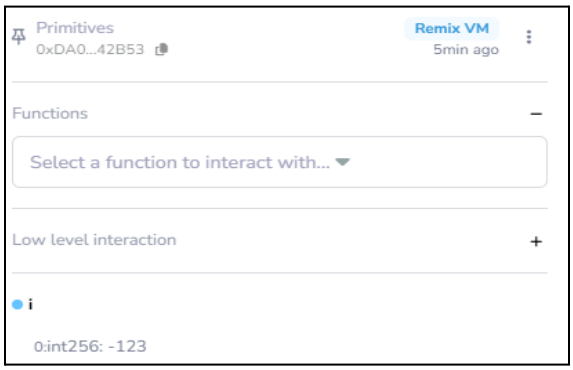




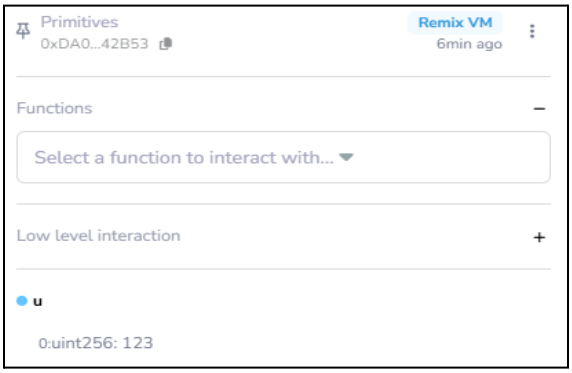
addr



bool

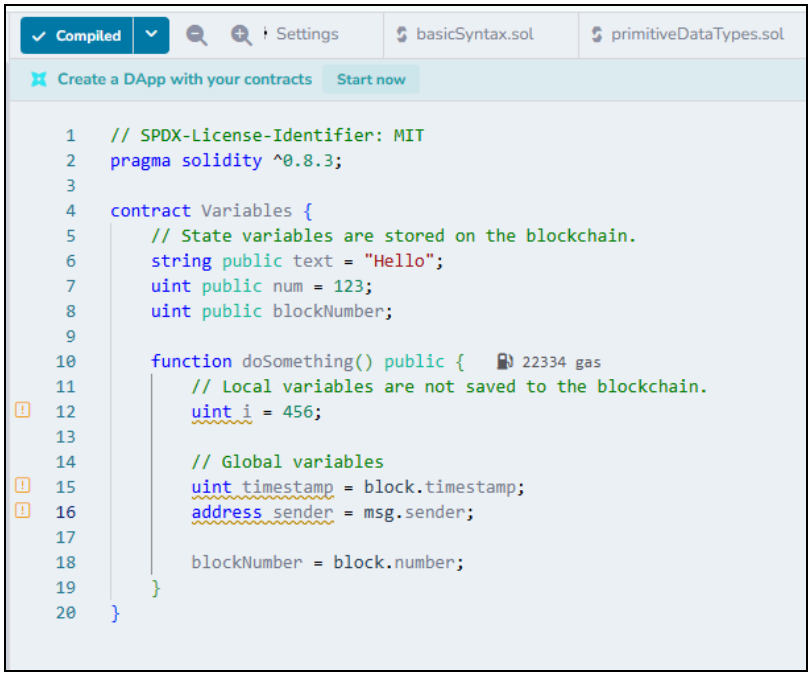


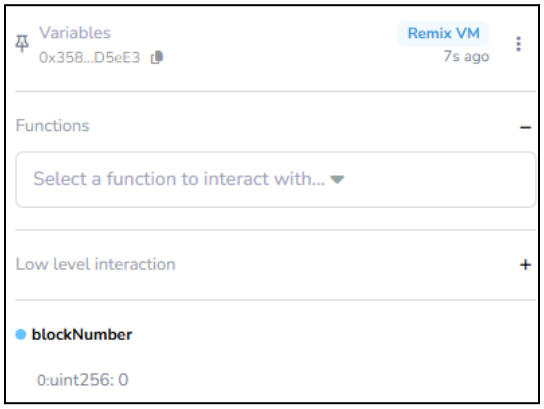
i



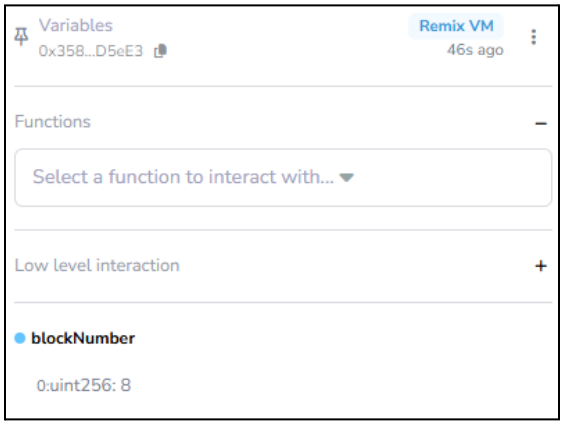
u

Deployed
Tutorial no. 4



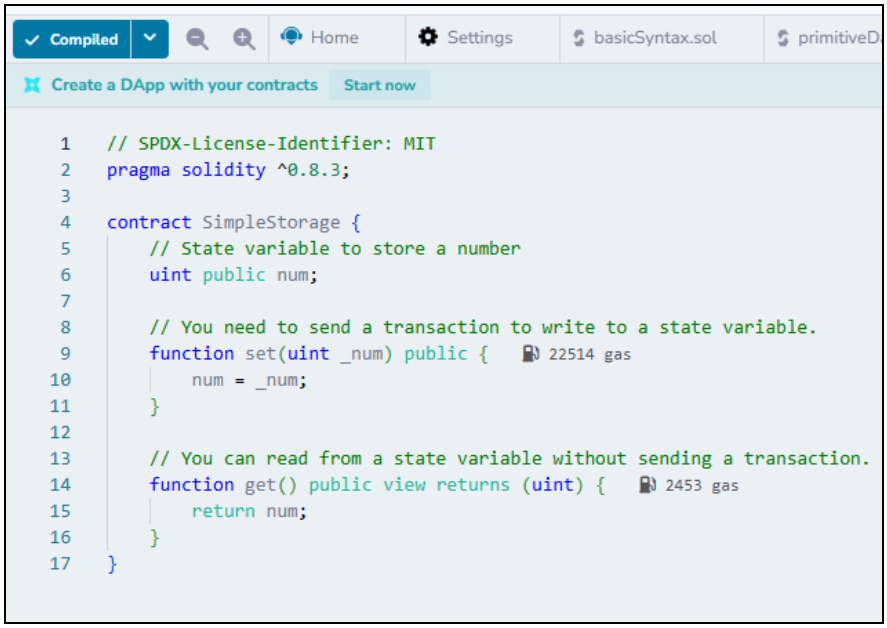


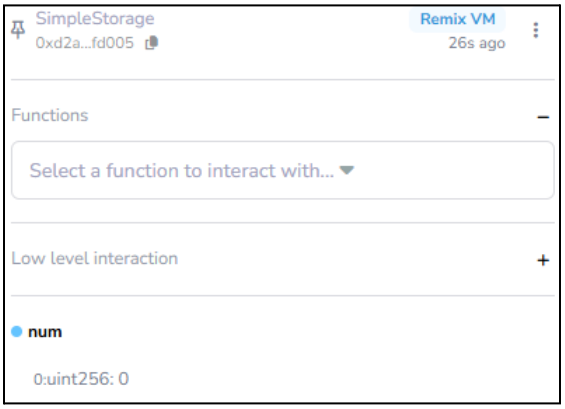
After doSomething



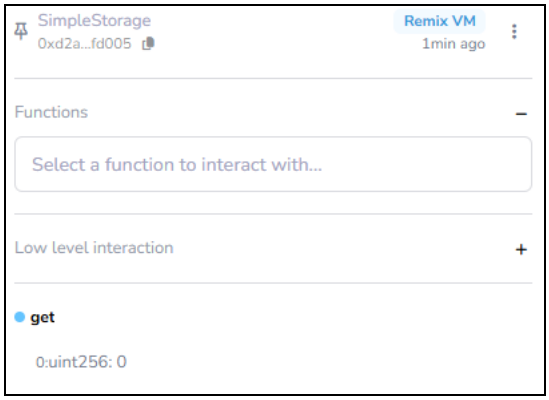
Deployed

Tutorial no. 5

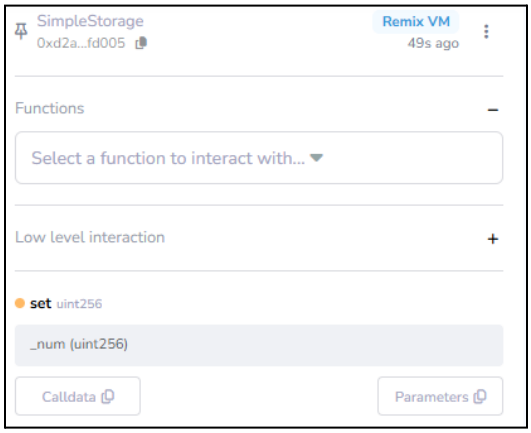




num



get



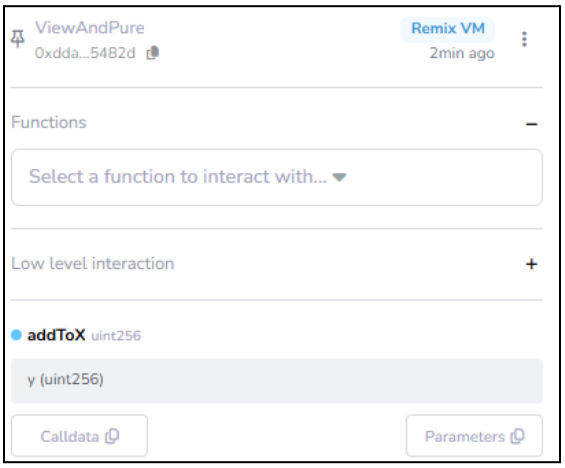
set

Tutorial no. 6

Compiled | Settings | basicSyntax.sol | primitiveDataTypes.sol | variab

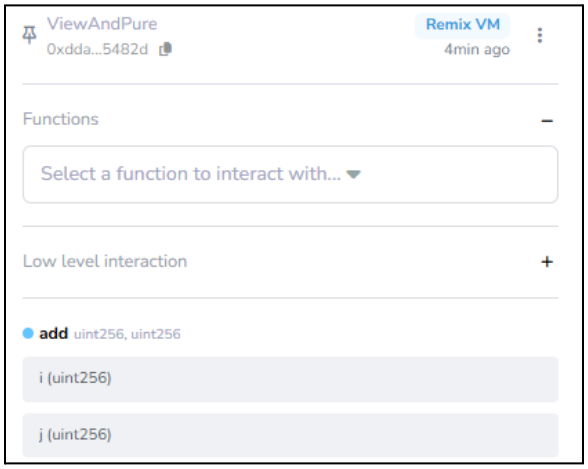
Create a DApp with your contracts Start now

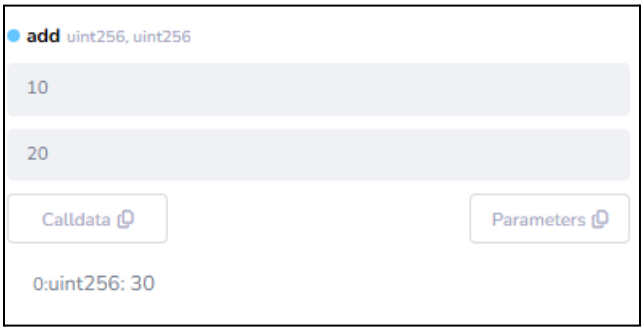
```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract ViewAndPure {
5      uint public x = 1;
6
7      // Promise not to modify the state.
8      function addToX(uint y) public view returns (uint) {    infinite gas
9          return x + y;
10     }
11
12     // Promise not to modify or read from the state.
13     function add(uint i, uint j) public pure returns (uint) {    infinite gas
14         return i + j;
15     }
16 }
```





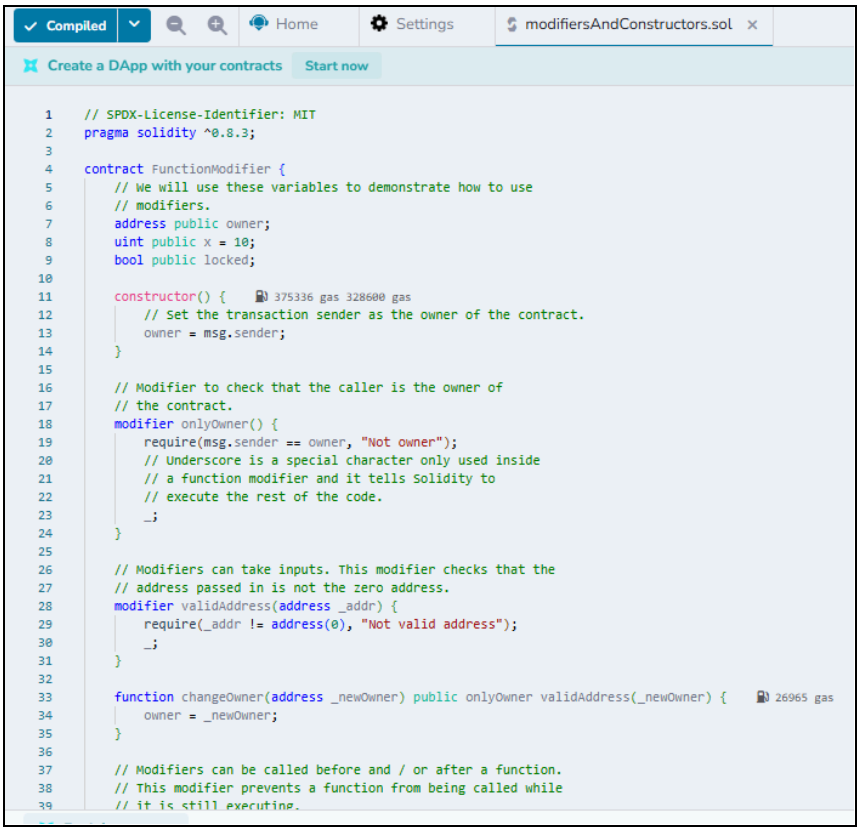
addToX

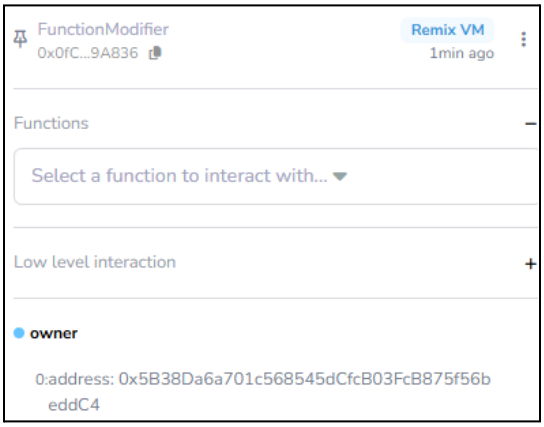


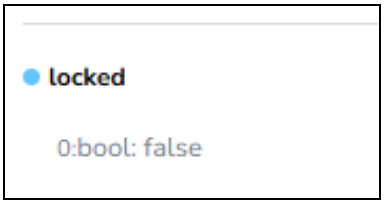


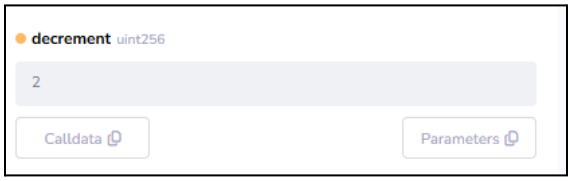
add

Tutorial no. 7





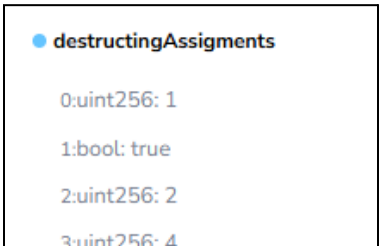


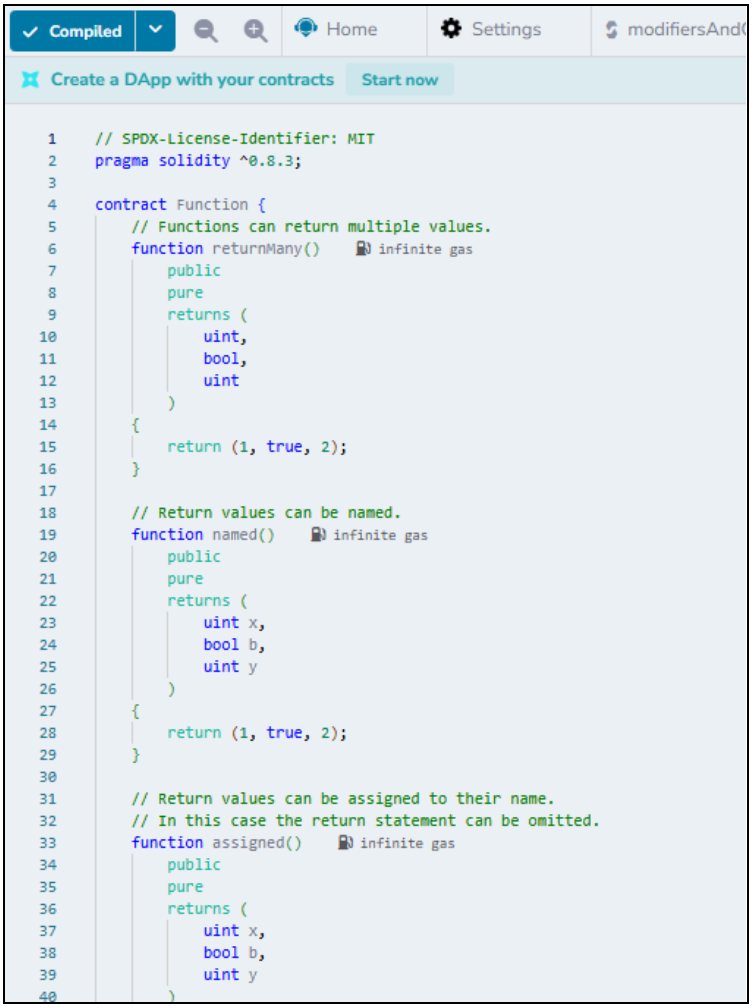


owner

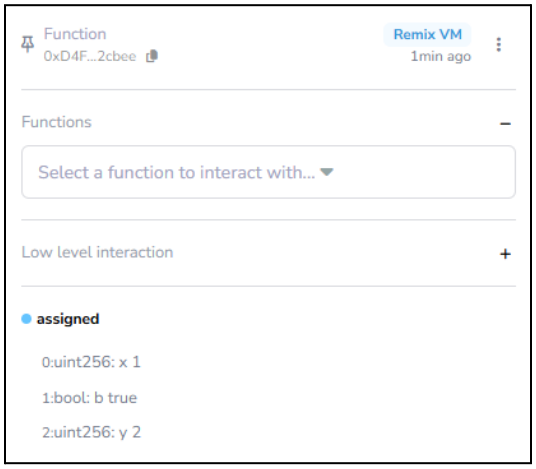


Tutorial no. 8



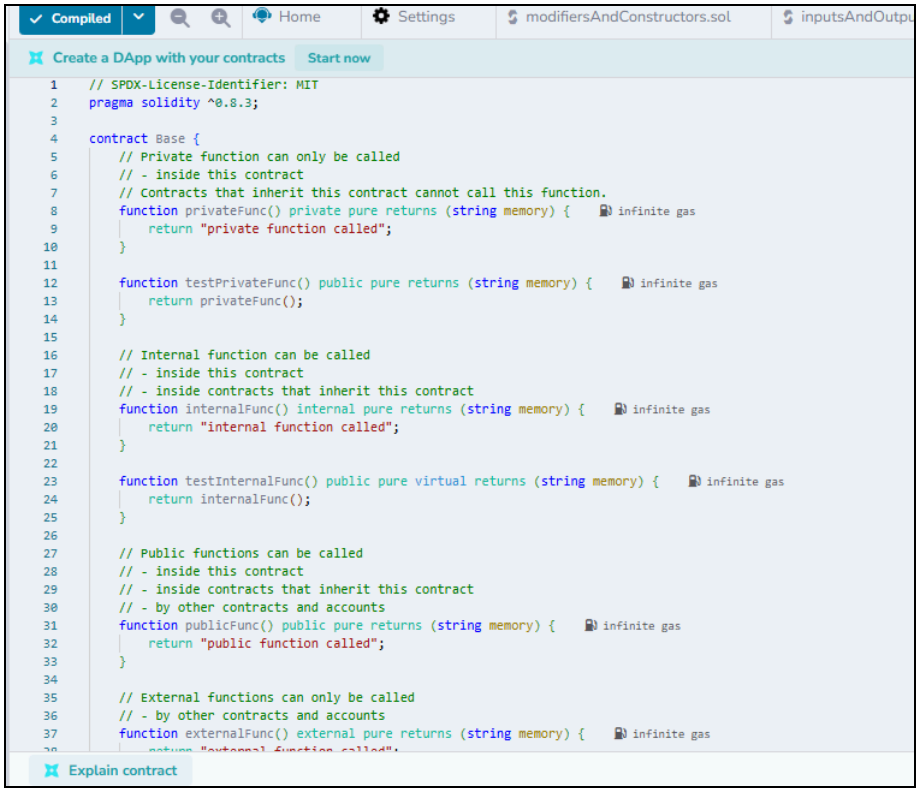


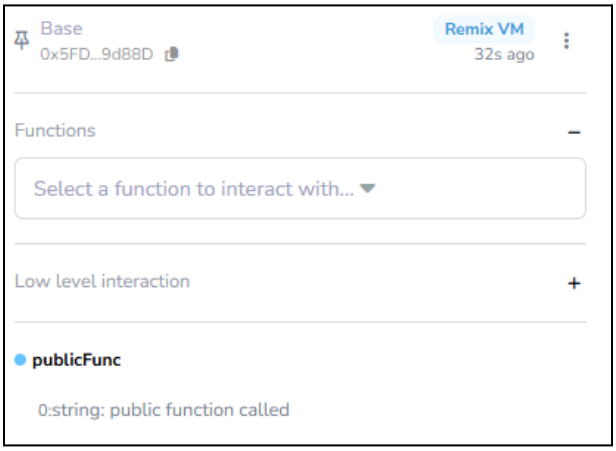


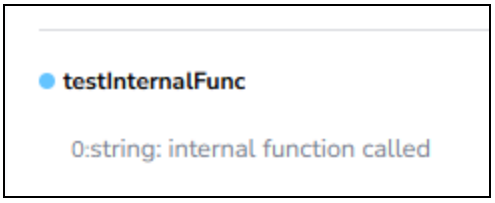




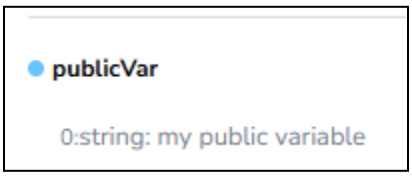
Tutorial no. 9


Compiled
Home
Settings
modifiersAndConstructors.sol
inputsAndOutpu
Create a DApp with your contracts
Start now
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Base {
// Private function can only be called
// - inside this contract
// Contracts that inherit this contract cannot call this function.
function privateFunc() private pure returns (string memory) {
infinite gas
return "private function called";
}
function testPrivateFunc() public pure returns (string memory) {
infinite gas
return privateFunc();
}
// Internal function can be called
// - inside this contract
// - inside contracts that inherit this contract
function internalFunc() internal pure returns (string memory) {
infinite gas
return "internal function called";
}
function testInternalFunc() public pure virtual returns (string memory) {
infinite gas
return internalFunc();
}
// Public functions can be called
// - inside this contract
// - inside contracts that inherit this contract
// - by other contracts and accounts
function publicFunc() public pure returns (string memory) {
infinite gas
return "public function called";
}
// External functions can only be called
// - by other contracts and accounts
function externalFunc() external pure returns (string memory) {
infinite gas
Explain contract


Base
0x5FD...9d88D
Remix VM
32s ago
Functions
Select a function to interact with...
Low level interaction
publicFunc
0:string: public function called

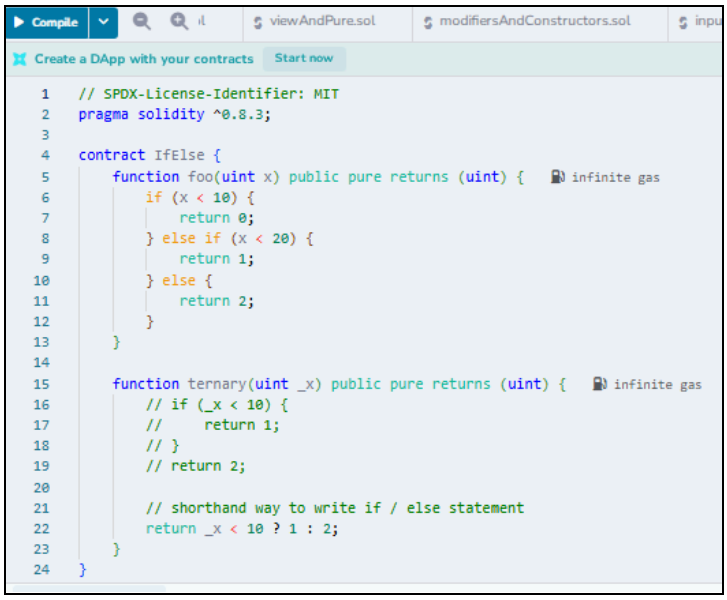

testInternalFunc
0:string: internal function called

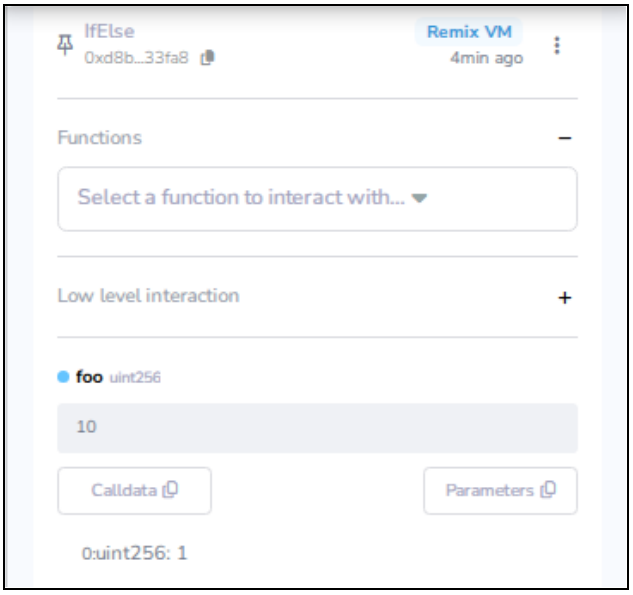

testPrivateFunc
0:string: private function called

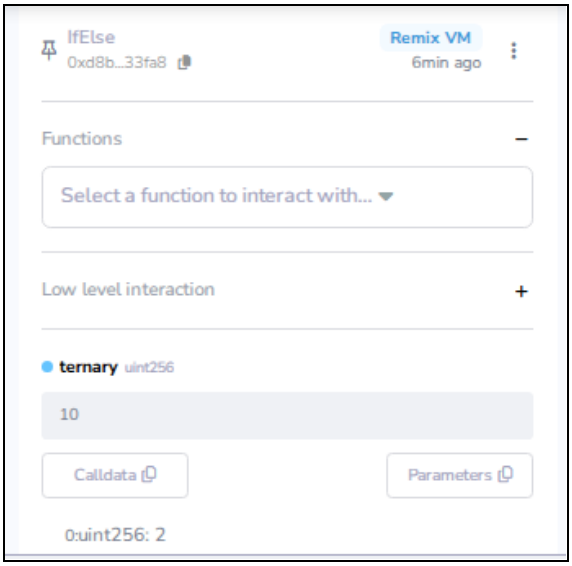

publicVar
0:string: my public variable


externalFunc
0:string: external function called

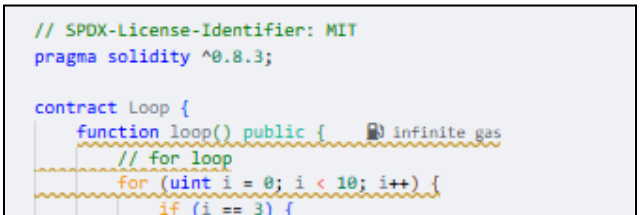
Tutorial 10

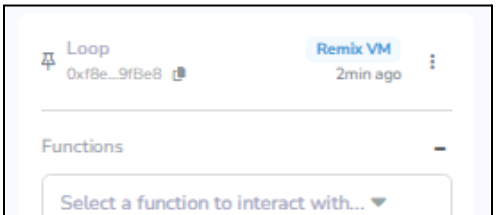

Compile
viewAndPure.sol
modifiersAndConstructors.sol
Create a DApp with your contracts
Start now
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract IfElse {
function foo(uint x) public pure returns (uint) {
infinite gas
if (x < 10) {
return 0;
} else if (x < 20) {
return 1;
} else {
return 2;
}
}
function ternary(uint _x) public pure returns (uint) {
infinite gas
// if (_x < 10) {
// return 1;
// }
// return 2;
// shorthand way to write if / else statement
return _x < 10 ? 1 : 2;
}
}


IfElse
0xd8b...33fa8
Remix VM
4min ago
Functions
Select a function to interact with...
Low level interaction
foo uint256
10
Calldata
Parameters
0:uint256: 1


IfElse
0xd8b...33fa8
Remix VM
6min ago
Functions
Select a function to interact with...
Low level interaction
ternary uint256
10
Calldata
Parameters
0:uint256: 2

Tutorial 11


// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract Loop {
function loop() public {
infinite gas
// for loop
for (uint i = 0; i < 10; i++) {
if (i == 3) {


Loop
0xf8e...9f8e8
Remix VM
2min ago
Functions
Select a function to interact with...

Tutorial 12

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

contract Array {
    // Several ways to initialize an array
    uint[] public arr;
    uint[] public arr2 = [1, 2, 3];
    // Fixed sized array, all elements initialize to 0
    uint[10] public myFixedSizeArr;

    function get(uint i) public view returns (uint) {    infinite gas
        return arr[i];
    }


    function getArr() public view returns (uint[] memory) {    infinite gas
        return arr;
    }

    function push(uint i) public {    46828 gas
        // Append to array
        // This will increase the array length by 1.
        arr.push(i);
    }

    function pop() public {    29462 gas
        // Remove last element from array
        // This will decrease the array length by 1
        arr.pop();
    }
```

```
    function getLength() public view returns (uint) {    2483 gas
        return arr.length;
    }

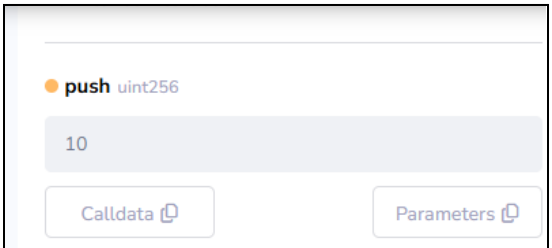
    function remove(uint index) public {    7689 gas

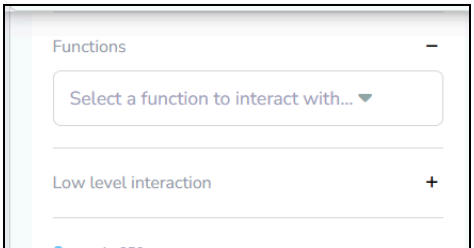
        delete arr[index];
    }
}

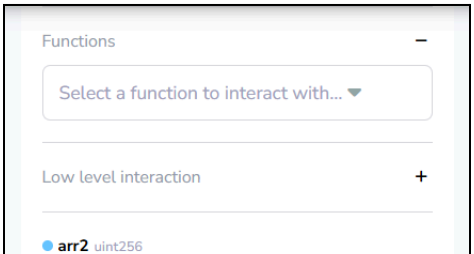
contract CompactArray {
    uint[] public arr;
    function remove(uint index) public {    infinite gas
        arr[index] = arr[arr.length - 1];
        arr.pop();
    }
    function test() public {    infinite gas
        arr.push(1);
        arr.push(2);
        arr.push(3);
        arr.push(4);
        // [1, 2, 3, 4]

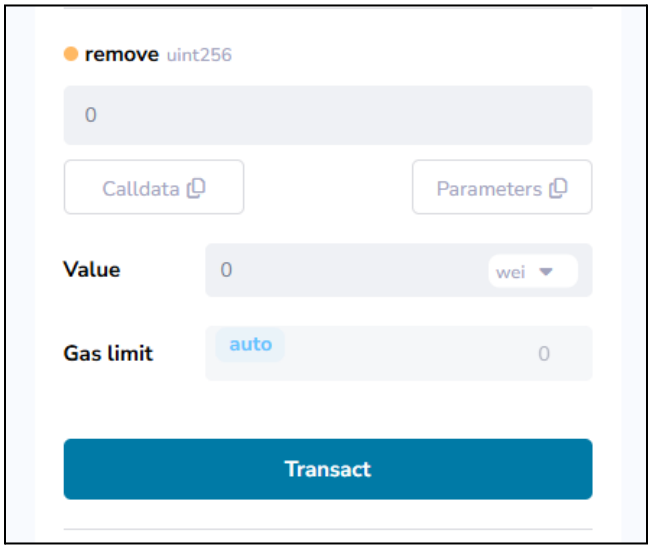
        remove(1);
        // [1, 4, 3]

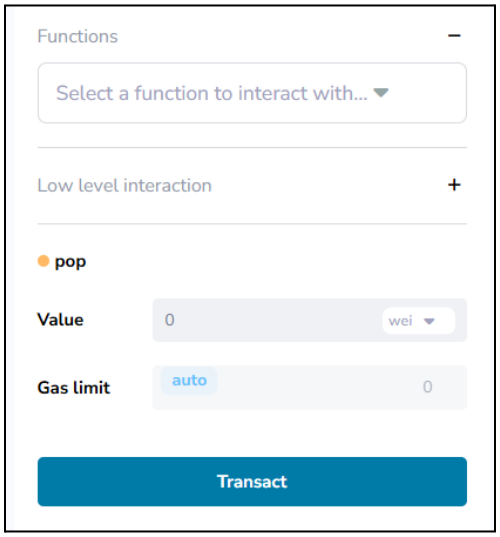
        remove(2);
        // [1, 4]
    }
}
```











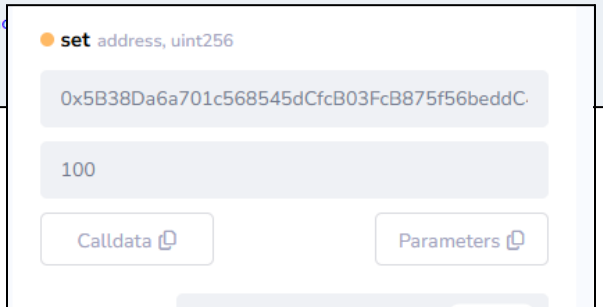
Tutorial 13

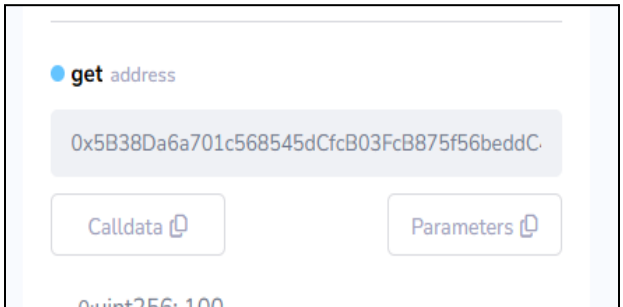
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

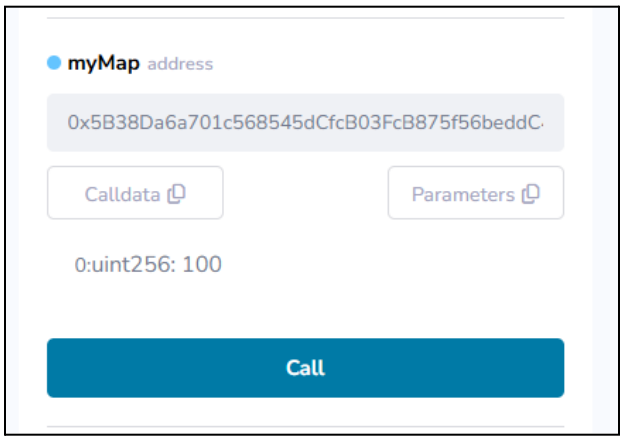
contract Mapping {
    mapping(address => uint) public myMap;
    function get(address _addr) public view returns (uint) {
        return myMap[_addr];
    }
    function set(address _addr, uint _i) public {
        myMap[_addr] = _i;
    }
    function remove(address _addr) public {
        delete myMap[_addr];
    }
}
contract NestedMapping {
    mapping(address => mapping(uint => bool)) public nested;

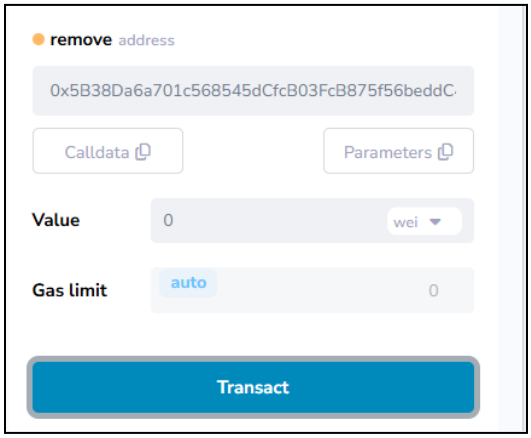
    function get(address _addr1, uint _i) public view returns (bool) {
        return nested[_addr1][_i];
    }
    func
```

```
    ) public {
        nested[_addr1][_i] = _boo;
    }
    function remove(address _addr1, uint _i) public {
        delete nested[_addr1][_i];
    }
}
```









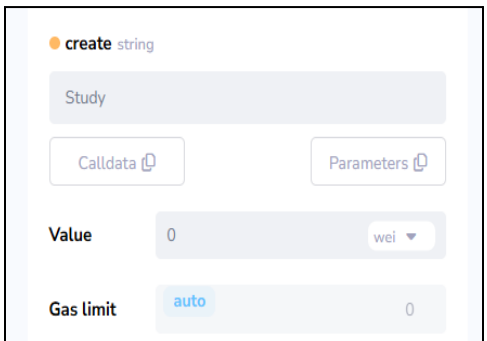
Tutorial 14

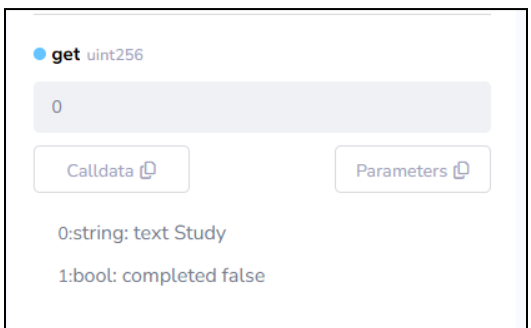
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;

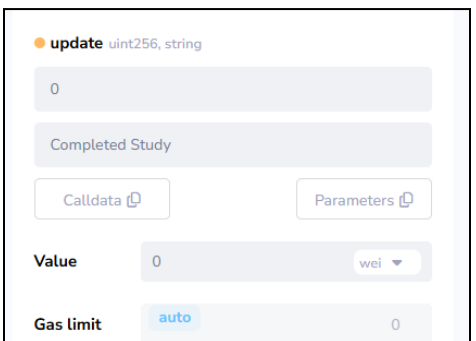
contract Todos {
    struct Todo {
        string text;
        bool completed;
    }
    Todo[] public todos;

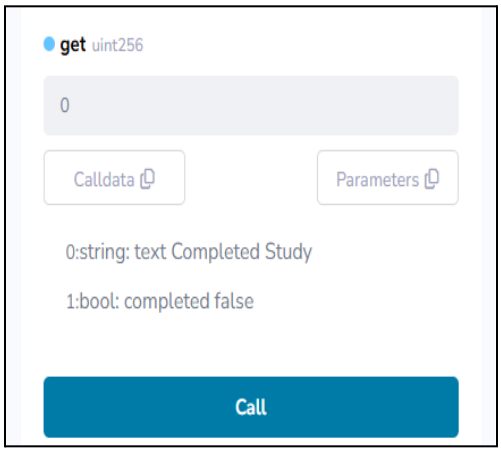
    function create(string memory _text) public {    infinite gas
        todos.push(Todo(_text, false));
        todos.push(Todo({text: _text, completed: false}));
        Todo memory todo;
        todo.text = _text;
        todos.push(todo);
    }
    function get(uint _index) public view returns (string memory text, bool completed) {
        Todo storage todo = todos[_index];
        return (todo.text, todo.completed);
    }
```

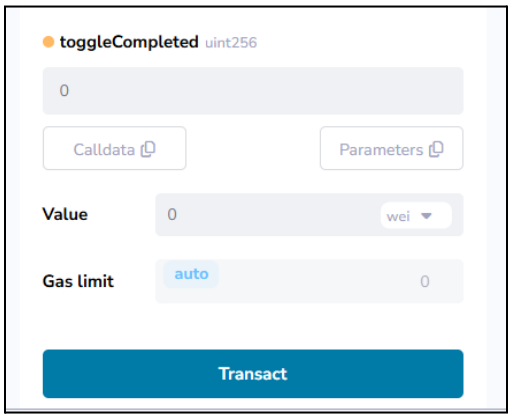
```
    function update(uint _index, string memory _text) public {
        Todo storage todo = todos[_index];
        todo.text = _text;
    }
    function toggleCompleted(uint _index) public {    28995 gas
        Todo storage todo = todos[_index];
        todo.completed = !todo.completed;
    }
}
```

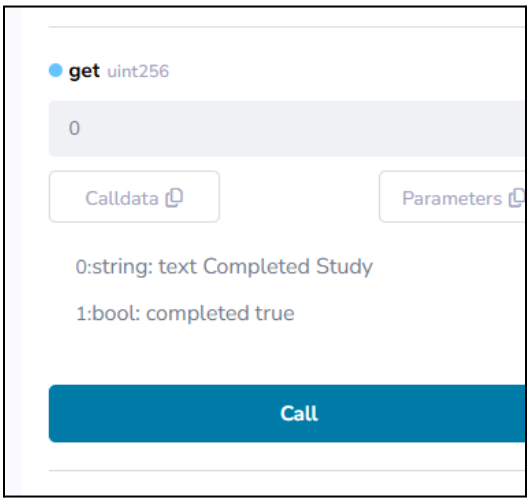




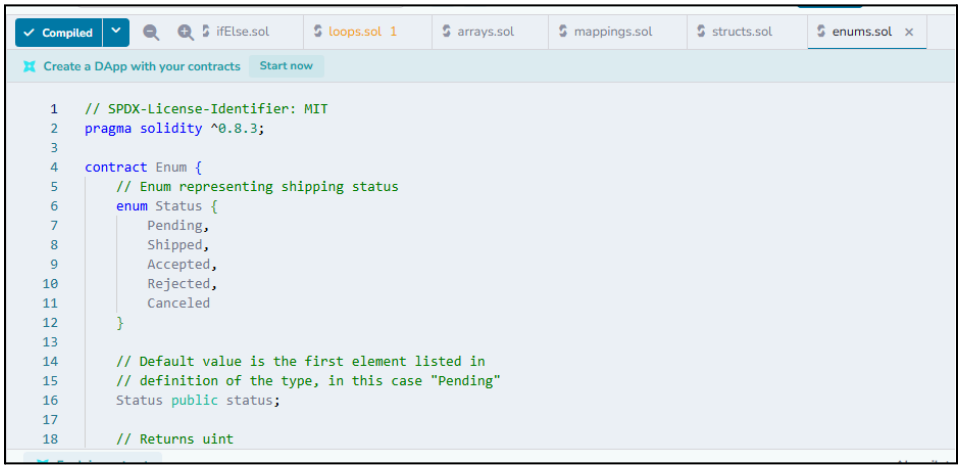


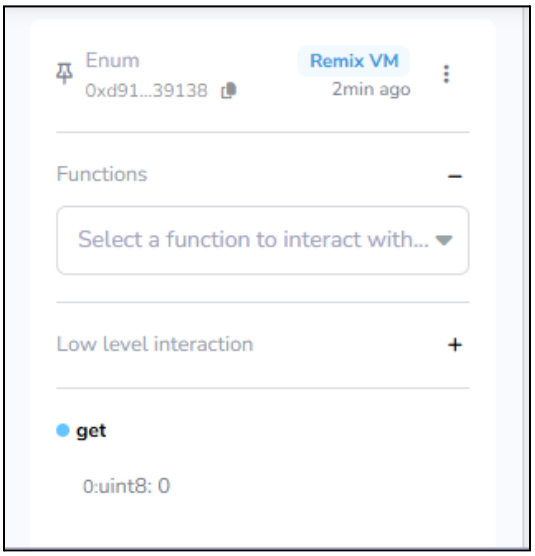






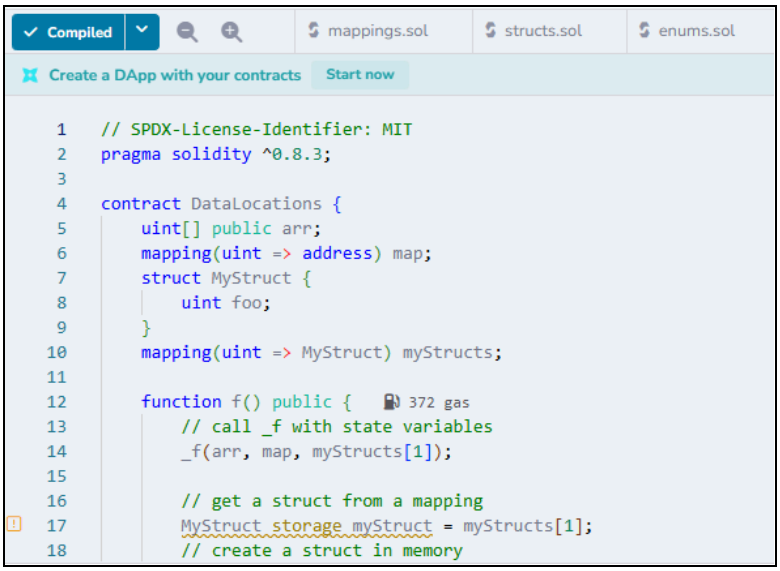
Tutorial 15

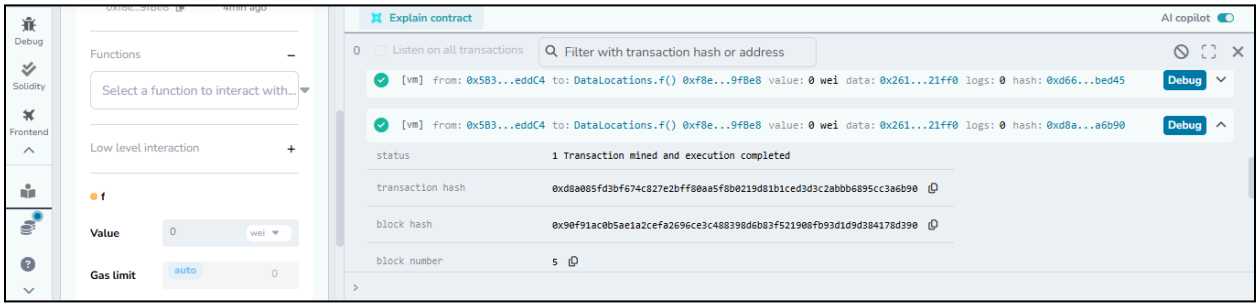




The getSize() function returned 0, representing the default enum value S.

Tutorial 16





The DataLocations smart contract was successfully compiled, deployed and executed in Remix VM. The f() function demonstrated the use of different data locations such as storage and memory.

Tutorial 17

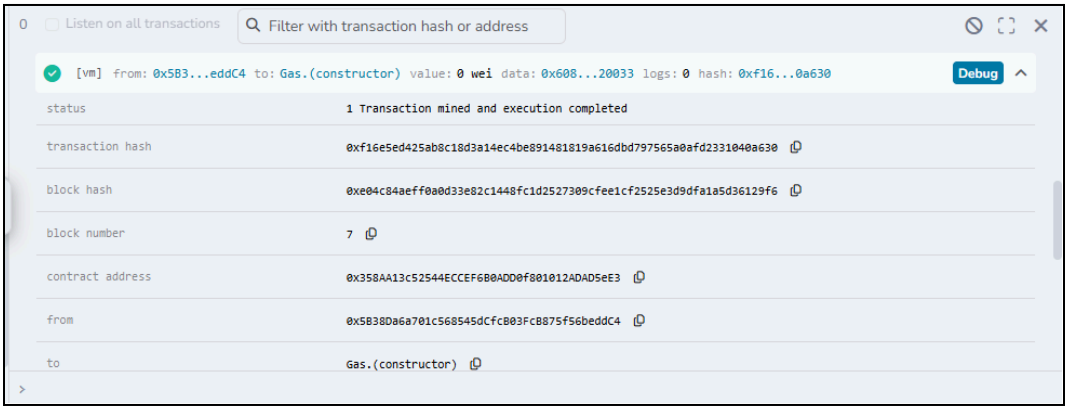




Ether is the main currency of Ethereum, while Wei is its smallest unit. Solidity provides wei, gwei, and ether units for handling Ether values in smart contracts. **1 Ether = 10^{18} Wei** and **1 Gwei = 10^{9} Wei**.

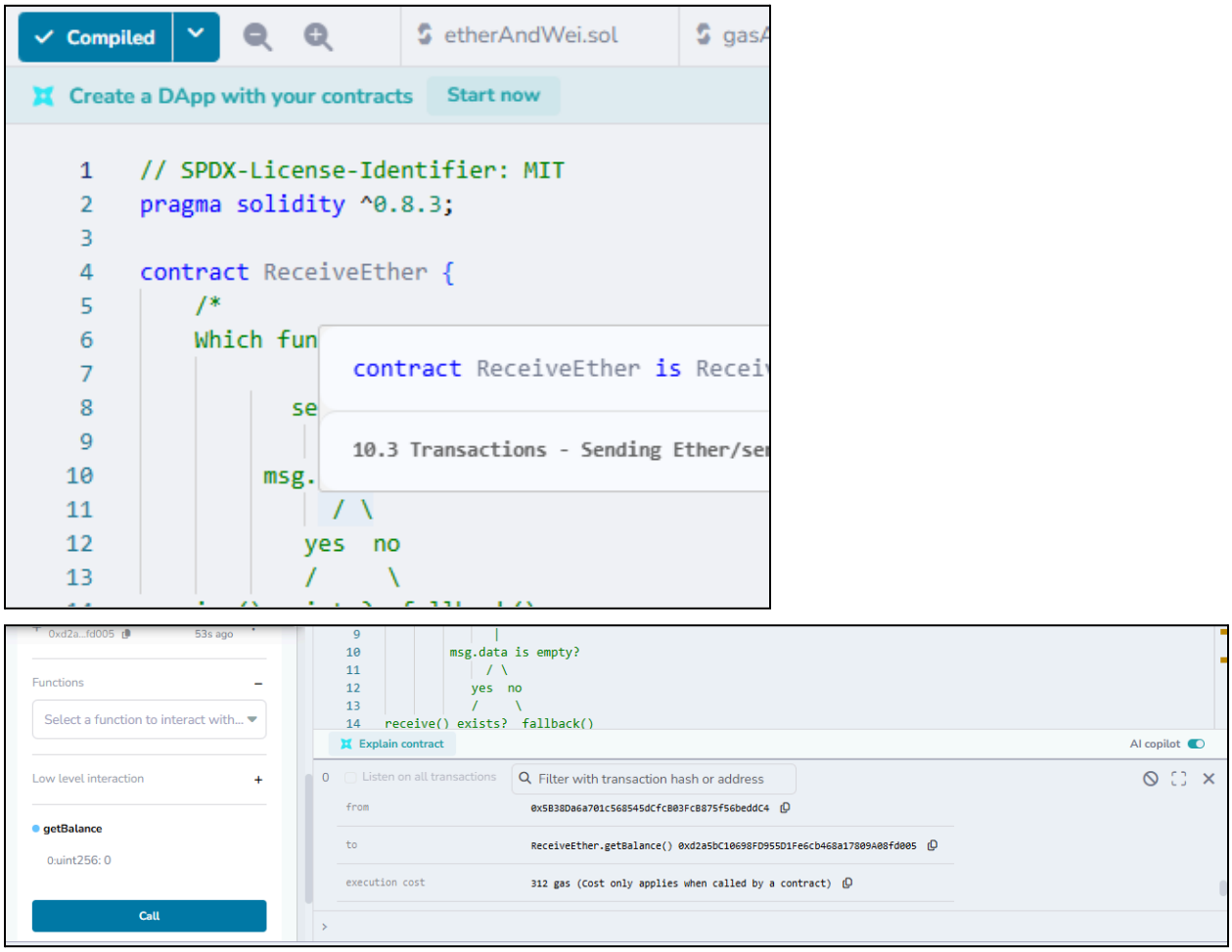
Tutorial 18





The `forever()` function contains an infinite loop that continuously consumes gas. The transaction eventually fails when the available gas is exhausted.

Tutorial 19



The ReceiveEther contract was successfully deployed and Ether was sent to the contract through a transaction, demonstrating Ether transfer and contract handling.

RESULT

Successfully compiled, deployed, and executed the Solidity programs demonstrating gas, Ether transfer, and other Ethereum concepts.

CONCLUSION

The experiment provided practical understanding of Solidity fundamentals and smart-contract execution using Remix IDE.